SKILL.md · References / Scripts / Assets Quick Reference Guide

THE CORE PRINCIPLE: When your SKILL.md approaches 500 lines, you have two choices — scope creep (do less) or extraction (move complexity outside the file). references/ · scripts/ · assets/ are the extraction architecture. They keep SKILL.md lean while giving your Skill unlimited depth on demand.

SECTION 1 · The Three Folders at a Glance — What each one is for and when to create it

Folder	Purpose	Loaded When?	Required For	Class
`references/`	Detailed documentation the Skill consults during execution. Policies, standards, lookup tables, deep-dive guides. Kept external to preserve SKILL.md token budget.	Phase 3 — Execution. Loaded on demand via <decision_criteria> directives. Only what is needed, when needed.	• Complex policies too long for SKILL.md • Reference tables (error codes, status maps) • Domain standards (SQL style, security rules) • Regulatory requirements	Class B — optional Class C — expected
`scripts/`	Executable code for automation and verification. The Skill's automated hands — it doesn't just advise, it acts and checks. Verification scripts are Anthropic's #1 recommendation.	Phase 3 — Execution. Invoked explicitly when decision logic calls for them. Not auto-loaded.	• verify.sh — confirm Skill output is correct • test_suite.py — regression testing • compare_results.py — before/after analysis • setup scripts, migration runners	Class C — essential (Class B if verification-critical)
`assets/`	Templates, images, data files the Skill uses or produces. Static resources the model references or fills in. Not executable — purely content.	Phase 3 — Execution. Referenced when Skill needs to use or produce a specific file format.	• Output templates (email, report, PR comment) • Example files (sample input, expected output) • Data files (lookup CSVs, config JSON) • Images for documentation	Class A/B/C (whenever templated output needed)

SECTION 2 · Folder Structure — How to organise your Skill directory

Class A — Simple

```
your-skill-name/
│
└── SKILL.md

That's it.
Single file only.
< 100 lines.

No references/
No scripts/
No assets/ needed.
```

Class B — Intermediate

```
your-skill-name/
│
├── SKILL.md
│
├── references/          ← optional
│   ├── standards.md
│   └── lookup-table.md
│
└── assets/              ← optional
    └── output-template.md
```

Class C — Advanced

```
your-skill-name/
│
├── SKILL.md
│
├── references/          ← expected
│   ├── domain-rules.md
│   ├── error-codes.md
│   └── policy.md
│
├── scripts/             ← essential
│   ├── verify.sh
│   └── test_suite.py
│
└── assets/
    ├── template.md
    └── sample-input.json
```

SECTION 3 · When to Extract — Decision guide for moving content out of SKILL.md

Situation	Signal	Extract To	How to Reference in SKILL.md	Why

Policy or standard is too long	Section in SKILL.md exceeds 50 lines and is mostly reference, not logic.	references/	`<decision_criteria>`
			`IF rule lookup needed:`
			`→ Read references/policy.md`
			`→ Apply relevant section`
			`</decision_criteria>`
				Keeps SKILL.md under 500 lines.
				Policy loaded only when needed.

Verification logic is complex	Self-verification section needs more than a checklist — it needs to run code.	scripts/	`<verification>`
			`Run: scripts/verify.sh`
			`Pass = exit code 0`
			`Fail = review error output`
			`</verification>`
				Automated verification is Anthropic's #1 recommendation.
				Scripts are reusable and testable.

Output needs a template	Skill produces formatted output (email, PR comment, report) with consistent structure.	assets/	`<action>`
			`Load assets/template.md`
			`Fill placeholders with task data`
			`Return completed document`
			`</action>`
				Separates content from logic.
				Template can be updated without touching Skill logic.

Examples are too long	Example section has 3+ before/after pairs with extensive context. SKILL.md getting heavy.	references/	`<example>`
			`See references/examples.md`
			`for full worked examples.`
			`</example>`
				SKILL.md stays lean.
				Examples loaded only when model needs them.

Domain knowledge is deep	Skill covers a specialised domain (SQL, security, legal) with extensive rules.	references/	`<decision_criteria>`
			`IF domain rule needed:`
			`→ Read references/domain-rules.md`
			`→ Apply matching rule`
			`</decision_criteria>`
				Domain knowledge grows without bloating SKILL.md.
				Loaded on demand only.

Data lookup required	Skill needs to look up values from a table (error codes, status maps, config).	references/
		or assets/
			`<action>`
			`Look up value in`
			`references/lookup-table.md`
			`</action>`
				Tables change independently of logic.
				Keep data and logic separate.

SECTION 4 · Naming Conventions — Consistent naming makes Skill directories readable and maintainable

Folder	Pattern	Good Examples	Bad Examples	Rule
references/	[topic]-[type].md	✓ sql-style-guide.md ✓ error-codes.md ✓ security-policy.md ✓ budget-rules.md	✗ ref1.md ✗ stuff.md ✗ REFERENCE_FILE_v2_FINAL.md	Kebab-case. Descriptive enough to know content without opening.
scripts/	[action].sh or [action].py	✓ verify.sh ✓ test_suite.py ✓ compare_results.py ✓ setup.sh	✗ script1.py ✗ run.sh ✗ myScript.py	Verb-first naming. Action is immediately clear.
assets/	[type]-template.md or [description].[ext]	✓ email-template.md ✓ pr-comment-template.md ✓ sample-input.json ✓ expected-output.csv	✗ template.md ✗ file1.json ✗ Asset_Final_v3.md	Include file type in name. Distinguish templates from data files.

SKILL.md · References / Scripts / Assets Quick Reference Guide (Page 2)

SECTION 5 · The 500-Line Rule & Common Anti-Patterns

THE 500-LINE RULE: When SKILL.md approaches 500 lines, stop and ask:

1. Can I reduce scope? (One Thing Test — remove the 'and') → Split into separate Skills
2. Can I extract? → Move reference content to references/ · Move verification to scripts/ · Move templates to assets/
3. Do I need multi-file architecture? → If yes, you have a Class C Skill. See Section 1.3 of the curriculum.

Anti-Pattern — What NOT to do			Fix — What to do instead
Embedding a 200-line policy document inside SKILL.md	SKILL.md bloats past 500 lines. Every activation loads content that is only needed occasionally.	→	Move policy to references/policy.md. Reference it conditionally in <decision_criteria>.
Writing verification as a prose checklist for a Class C Skill	Manual checklists are skipped or inconsistently applied. Anthropics #1 recommendation is automated verification.	→	Write scripts/verify.sh. Call it from the <verification> section. Exit code 0 = pass.
Storing output templates inline in SKILL.md	Template changes require editing Skill logic. Data and logic are entangled.	→	Move template to assets/template.md. Skill loads and fills it during execution.
One massive references/ file for everything	Model loads entire file when it only needs one section. Token waste. Slow. Hard to maintain.	→	Split into topic-specific files. error-codes.md · security-policy.md · style-guide.md

✦ PROGRESSIVE DISCLOSURE — Where references/ scripts/ assets/ fit in the loading lifecycle

Phase	What Loads	Token Cost	includes references/ scripts/ assets/?	Notes
Phase 1 Skill Loads	YAML frontmatter only. name + description.	~50–100 tokens	✗ No	Identity card. Discovery only.
Phase 2 Skill Activates	Full SKILL.md markdown body. All 8 core components.	~500–5,000 tokens	✗ No (only references in <decision_criteria>)	Instructions enter context. references/ paths are noted but not yet loaded.
Phase 3 Task Executes	references/ · scripts/ · assets/ Loaded on demand as needed.	Only what is called when it is called	✓ **Yes — this is their phase**	Maximum efficiency. Deep knowledge available without context bloat.